# Problem Set 3

Due Wednesday October 7, 10 am

## Comments

- This covers material in Unit 4.
- It's due at 10 am (Pacific) on Wednesday October 7, both submitted as a PDF to Pensive as well as committed to your GitHub repository.

## Formatting requirements

1. Your electronic solution should be in the form of an Quarto file named `ps3.qmd`, with Python code chunks. If you want to initially work in a Jupyter notebook, that is fine, but you'll need to run `quarto convert file.ipynb` to generate a qmd file before rendering to PDF and submitting.
2. Your solution should not just be Python code - you should have text describing how you approached the problem and what the various steps were. Your code should have comments indicating what each function or block of code does, and for any lines of code or code constructs that may be hard to understand, a comment indicating what that code does.
3. You do not need to (and should not) show exhaustive output, but in general you should show short examples of what your code does to demonstrate its functionality. Please see the grading rubric, and note that the output should be produced as a result of the code chunks being run during the rendering process, not by copy-pasting of output from running the code separately (and definitely not as screenshots).

## Problems

1. This problem explores how Python stores strings (and more about lists), which we saw some in class.

   a. Let's consider the following lists of strings. Determine what storage is reused (if any) in storing 'abc' in the two lists.

   ```
   a = ['abc', 'xyz', 'def', 'ghi']
   b = ['abc']*4
   ```

   b. Next, let's dig into how much memory is used to store the information in a list of strings. Determine (i) how much memory is used to store a simple string (and how this varies with the length of the string), including any metadata, (ii) how much memory is used for metadata

of the list object, and (iii) how much is used for the references to the actual elements of the list (i.e., how the size of the list grows with the number of elements). Experimenting with lists of different lengths and strings of different lengths should allow you to work this out from examples without having to try to find technical documentation for Python's internals. In your solution, show the examples you used to determine your answers.

2. Consider multiplying an arbitrary $n \times n$ matrix $X$ and a diagonal $n \times n$ matrix $D$.

   a. How many multiplications are done if you simply matrix multiply $D$ and $X$?
   b. In principle, how many multiplications need to be done to obtain the result without doing unnecessary calculations?
   c. How can I use `numpy` operations to compute $XD$ efficiently (without any use of loops)?
   d. How can I use `numpy` operations to compute $DX$ efficiently (without any use of loops)?

3. This problem asks you to efficiently compute a somewhat complicated log likelihood, arising from a computation from a student's PhD research. The following is the probability mass function for an overdispersed binomial random variable:

$$P(Y = y; n, p, \phi) = \frac{f(y; n, p, \phi)}{\sum_{k=0}^{n} f(k; n, p, \phi)}$$

$$f(k; n, p, \phi) = \binom{n}{k} \frac{k^k (n-k)^{n-k}}{n^n} \left( \frac{n^n}{k^k (n-k)^{n-k}} \right)^{\phi} p^{k\phi} (1-p)^{(n-k)\phi}$$

   where the denominator of $P(Y = y; n, p, \phi)$ serves as a normalizing constant to ensure this is a valid probability mass function.

   We'll explore how would one efficiently code the computation of the denominator. For our purposes here you can take $n = 10000$, $p = 0.3$ and $\phi = 0.5$ when you need to actually run your code. Recall that $0^0 = 1$.

   a. Write a basic version using map operations, where you have a function that carries out a single calculation of $f$ for a value of $k$ and then use `map` to execute it for all the elements of the sum. Make sure to do all calculations on the log scale and only exponentiate before doing the summation. This avoids the possibility of numerical overflow or underflow that we'll discuss in 243B in Unit 8.
   b. Now create a vectorized version using numpy arrays. Compare timing to the basic non-vectorized version.
   c. Use timing and profiling tools to understand what steps are slow and try to improve your efficiency. Keep an eye out for repeated calculations and calculations/operations that don't need to be done. Compare timing to your initial vectorized version in (b).

4. SQL practice using the Stack Overflow database.

   a. Find all the users (including their `displayname`) who have answered but never asked a question. You're welcome to set up your solution using views, but you should not need to (apart from dealing with the NULL ownerid values mentioned below). In your solution, provide as many separate queries as needed to illustrate **all** of the following SQL functionality: (i) a subquery, (ii) a set operation using INTERSECT, UNION or EXCEPT, and (iii) an

outer join (note that SQLite does not have a right outer join). Check the count of the number of results from the different queries to make sure they are the same.

Note that for reasons I haven't looked into, both the questions and answers tables contain rows where the `ownerid` is `NULL`. These can mess up certain versions of the queries I'm asking you to create. So first create views that omit those cases from those two tables. (One could do that within the main queries, but this is a good situation for using views to make the main queries easier to understand.)

b. Do the different approaches vary much in terms of how much time they take? What about using SQLite versus DuckDB?

c. See if you can get faster query times by setting up appropriate indexes.